Multi-Solution Search and the Limits of Neural Guidance in Rubik's Cube Solving

Akash Chakraborty and Atal Chaudhuri

Abstract We extend Kociemba's Two-Phase Rubik's Cube solver with multi-solution Phase 1 search, a time-bounded anytime mode, and neural move ordering. On 200 random cubes, multi-solution search (K=5) cuts the mean solution by 0.53 moves and raises $\leq$20-move solutions from 28% to 57.5%. Neural ordering, even after distillation into a 576 KB lookup table, yields only 1.13$\times$ node reduction—Kociemba's pruning tables already provide near-optimal ordering, leaving little room for learned heuristics. All code, data, and models are provided for reproducibility.

Keywords: Rubik's Cube, two-phase algorithm, Kociemba, IDA*, pruning tables, neural heuristics, move ordering, anytime solver

1 Introduction

1.1 The Rubik's Cube as a Search Problem

The Rubik's Cube, invented by Ernő Rubik in 1974, is one of the most widely studied combinatorial puzzles in computer science [22]. A standard 3×3×3 cube has 26 visible sub-cubes ("cubies"): 8 corners, 12 edges, and 6 face centres. Each corner shows 3 coloured stickers and each edge shows 2, giving 54 stickers in total.

The six faces (Up, Down, Left, Right, Front, Back) can each be turned in three ways—clockwise, counter-clockwise, or a half-turn—yielding 18 possible moves at every step. The number of reachable configurations is

Akash Chakraborty
Department of Computer Science and Engineering, Sister Nivedita University, Kolkata, e-mail: akashchakraborty694@gmail.com

Atal Chaudhuri
Department of Computer Science and Engineering, Sister Nivedita University, Kolkata, e-mail: atalc23@gmail.com

$$|G| = \frac{8! \cdot 3^8 \cdot 12! \cdot 2^{12}}{12} \approx 4.33 \times 10^{19}. \tag{1}$$

This enormous state space rules out any brute-force approach.

1.2 God's Number and Practical Solving

"God's Number" is the fewest moves needed to solve any configuration from the worst case. In the *half-turn metric* (HTM), where each quarter-turn or half-turn counts as one move, God's Number is **20**, proved by Rokicki et al. [2] in 2010. In other words, every scrambled cube *can* be solved in at most 20 moves—but finding those 20 moves requires searching an astronomically large space.

Kociemba's Two-Phase Algorithm [1] offers a practical middle ground. It does not guarantee exactly 20 moves for every cube, but it reliably produces 19–22 move solutions in under a second. Although Kociemba's solver is a mature benchmark, two observations motivate this paper. First, its default single-endpoint Phase 1 search accepts the first Phase 1 result it finds, leaving substantial solution-quality headroom unexplored. Second, recent neural-guided search methods such as DeepCubeA [4] achieve strong results, but at the cost of days of GPU training, hundreds of thousands of parameters, and per-node inference that does not transfer back to lightweight classical solvers. A practitioner who needs near-optimal solutions on a laptop—for teaching tools, interactive applications, or robotics—is caught between the two extremes.

1.3 Research Questions and Contributions

This motivates three concrete research questions:

RQ1
Can a simple algorithmic extension of Kociemba's solver—exploring multiple Phase 1 endpoints—yield measurable improvements in solution quality without re-training or changing the underlying heuristic?

RQ2
Does cube-state representation materially affect the accuracy of a learned move predictor attached to a classical IDA* solver, and does any resulting accuracy gain translate into fewer expanded search nodes?

RQ3
Can the per-node overhead of neural inference be eliminated via lookup-table distillation, and is that sufficient to make learned move ordering competitive with Kociemba's existing pruning-table-based ordering?

We answer these questions with the following contributions:

1. **Multi-solution Phase 1 search** ($K > 1$): instead of accepting the first intermediate result, the solver explores K alternatives and keeps the shortest combined solution. On 200 random cubes this cuts mean moves by 0.53 (from 20.77 to 20.23) and raises $\leq$20-move solutions from 28% to 57.5%, at the cost of an $\approx$8.6$\times$ increase in expanded nodes (RQ1). This is the practical, easily deployable contribution of the paper.
2. **Representation affects raw move-prediction accuracy, but not solver node count (negative result).** We train an MLP move predictor under three regimes: (a) 9 scalar features ($\sim$5.6% proxy-label accuracy, essentially random), (b) 161 binned coordinate features with 100K proxy labels ($\sim$10–13% validation accuracy, three architectures), and (c) 161 binned coordinate features with 2M pruning-greedy labels (43.9% native top-1, 64.5% top-3; depth-stratified from 89.0% at depth $\leq$5 down to $\approx$30% at mid-depth). Across all three regimes the end-to-end node reduction over plain K=5 multi-solution search is within noise ($\leq$1.14$\times$), and the retrained LUT variant actually solves one fewer cube within the 5 s wall-clock budget (RQ2).
3. **LUT distillation works as intended, but cannot rescue a heuristic the solver does not need.** Distilling the trained network into a 32K-entry, 576 KB lookup table drops per-node inference cost to $\approx$1.5 μs, eliminating the overhead that Python-native inference imposes. In our 200-cube benchmark this removes the inference-cost barrier but still delivers only a 1.13$\times$ node reduction and *no* wall-clock speedup over non-neural K=5 search. A direct headroom measurement (Section 5.9) shows the gap between the default and oracle move orderings is large ($p_{\text{top}} - p_{\text{lex}} = 0.456$ on 6,000 walk-sampled states, implying a theoretical $\sim$10$\times$ ceiling), and a controlled h-delta retraining experiment shows that our lightweight 161-dim-binned-coordinate predictor cannot reach it—the model converges to the marginal class prior and matches the lexicographic baseline exactly: *the* 1.13$\times$ *wall is a feature-resolution ceiling, not an information-theoretic one* (RQ3).
4. **Time-bounded anytime mode** for interactive use, together with an analysis of the "admissible bound paradox" (Observation 1) explaining why a learned lower bound multiplies IDA* iteration cost, an order-statistic argument (Proposition 1) for why multi-solution Phase 1 helps, and an empirically calibrated cost model (Observation 2) for when neural move ordering can and cannot pay for itself.

All numbers reported in this paper are taken directly from the frozen artifacts `results/benchmark_200_retrained.json`, `results/arch_comp_replay_20260413.json`, and `results/prun_greedy_2M_meta.json`, shipped with the code.

1.4 Paper Organisation

Section 2 reviews prior work. Section 3 gives the mathematical background. Section 4 describes our methodology. Sections 5–6 present and discuss results. Section 7 concludes.

2 Related Work

Optimal solvers. Korf [5] built the first optimal Rubik's Cube solver using IDA* [12] with pattern databases [15], later extended to disjoint [16] and additive [10] variants. Rokicki et al. [2] proved God's Number is 20 using 35 CPU-years of computation. Demaine et al. [28] established worst-case bounds for generalised Rubik's Cubes.

Two-phase methods. Thistlethwaite [6] introduced nested-subgroup reduction with four phases. Kociemba [1] simplified this to two phases with coordinate-based pruning tables, dramatically improving speed. Human speedsolving methods like CFOP [11] use a similar multi-phase strategy but rely on pattern recognition rather than exhaustive search. Our work extends Kociemba's algorithm with multi-solution search and neural ordering.

Deep learning for the cube. DeepCubeA [4] uses deep reinforcement learning with weighted A* to find optimal solutions in 60% of cases, but requires $\sim$2 days of GPU training and $\sim$10 s per solve. Brunetto and Trunda [7] showed that feature representation is critical for distance estimation, consistent with our findings. More recently, Zawalski et al. [30] systematically compared hierarchical and flat planners on combinatorial puzzles and identified when learned heuristics justify their overhead—a question we answer in the negative for pruning-table-dominated search. Khandelwal et al. [31] proposed foundation-model heuristics that generalise across puzzle domains; our LUT distillation is complementary, showing that even a strong learned heuristic must clear the classical baseline's ordering quality to deliver end-to-end gains. More broadly, neural-guided tree search has been transformative in games like Go [17, 18].

Neural heuristics in planning. Shen et al. [8] showed that small heuristic errors cause exponential node growth in IDA*—the "heuristic noise" problem that our Observation 1 formalises. Arfaee et al. [19] proposed bootstrapped learning of heuristic functions for large state spaces. Ferber et al. [9] demonstrated the importance of domain-specific features for neural heuristics, supporting our finding that binned coordinate features dramatically outperform raw scalars.

Positioning. Unlike end-to-end learned solvers, we study neural heuristics as *add-ons* to an established classical algorithm, inspired by move-ordering techniques in game-tree search [20, 21]. To our knowledge, we are the first to (i) demonstrate that neural move ordering *saturates* due to pruning-table dominance in Kociemba's solver, quantifying the precise headroom ($p_{\text{top}} - p_{\text{lex}} = 0.456$) and the feature-resolution ceiling that prevents lightweight models from reaching it; (ii) distil a trained move predictor into an $O(1)$ lookup table for classical IDA* search; and

(iii) show that multi-solution Phase 1 search alone recovers most of the quality gap that neural methods target, at zero training cost.

3 Mathematical Foundation

3.1 Coordinate System

Rather than storing full permutation arrays, the solver encodes each cube state using a small number of *coordinates*—integers that capture the essential structure:

- **Twist** (0–2186): how the 8 corners are oriented.
- **Flip** (0–2047): how the 12 edges are oriented.
- **Slice** (0–494): which 4 of the 12 edge positions hold middle-layer edges.
- **URFtoDLF** and **URtoDF**: corner and edge permutation coordinates used in Phase 2.

The key advantage of coordinates is speed: applying a move just requires looking up $c' = \text{moveTable}[c][m]$, an $O(1)$ table lookup.

3.2 How the Two-Phase Algorithm Works

The algorithm solves the cube in two stages (Fig. 1):

Phase 1 searches for a sequence of moves that brings the cube from its scrambled state into a special subgroup

$$G_1 = \langle U, D, L^2, R^2, F^2, B^2 \rangle. \tag{2}$$

A cube is in G_1 when all edges are correctly oriented (flip $= 0$), all corners are correctly oriented (twist $= 0$), and the four middle-layer edges are in middle-layer positions (slice $= 0$). Phase 1 uses all 18 moves.

Phase 2 takes the cube from the G_1 state to the solved state (the identity). Only 10 moves are allowed (the generators of G_1), so the effective branching factor drops from 18 to 10 and stronger pruning tables apply. Phase 2 is typically much faster than Phase 1.

Both phases use **IDA*** (Iterative Deepening A*) [12], a memory-efficient extension of A* [13]. IDA* performs repeated depth-first searches with an increasing cost limit. At each node, a heuristic lower bound h [14] (read from precomputed *pruning tables*) estimates the remaining moves. If the current depth g plus h exceeds the limit, the branch is pruned. The critical property for our analysis: the number of nodes grows *exponentially* with the depth limit, so adding even one extra level of search multiplies the work by $\sim$13.

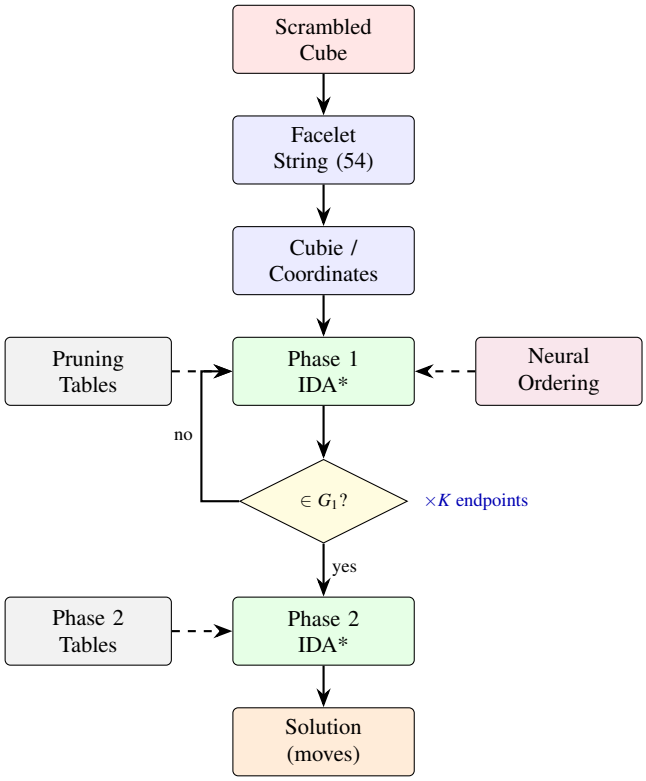

Fig. 1: Solver pipeline. The scrambled cube is converted to coordinates, then Phase 1 searches for states in subgroup G_1 (optionally guided by neural ordering). Phase 2 completes the solve. With $K > 1$, multiple Phase 1 endpoints are tried and the shortest overall solution is kept.

3.3 Pruning Tables

Pruning tables store the minimum moves needed to reach the target (the G_1 subgroup in Phase 1, or the identity in Phase 2) from each coordinate value. They are built once by backward breadth-first search and cached on disk ($\sim$20 MB). The heuristic at each node is the maximum of two independent lower bounds (e.g., Slice–Twist and Slice–Flip for Phase 1), which tightens the estimate while remaining admissible.

3.4 Consecutive Move Pruning

At each node the solver considers all 18 moves, but two simple rules eliminate redundant branches without losing solutions: (1) never apply two moves of the *same*

face in a row (e.g., U then U' is just $U2$ or the identity), and (2) for opposite faces that commute (U/D, L/R, F/B), impose a fixed ordering to avoid duplicates ($UD = DU$). These rules reduce the average branching factor from 18 to about 13.35—a $\sim 500\times$ reduction in tree size at depth 20.

3.5 Formal Results

We now state three results that combine analytical reasoning with empirical calibration—one observation about learned lower bounds in IDA*, one order-statistic proposition about multi-solution search, and one cost model for learned move ordering. Each statement is accompanied by a derivation sketch and empirical verification; they do not claim the generality of formal theorems but are predictive in our setting, and their quantitative predictions are confirmed by our experiments (Sections 5–6).

Observation 1 (Admissible Bound Paradox) *Let h^* be the optimal admissible heuristic for IDA*, and let $\hat{h} = h^* + \varepsilon$ be a learned heuristic that is* not *guaranteed admissible, overestimating on some states by up to $\varepsilon > 0$ moves. Then, in the worst case, IDA* with $\hat{h}$ expands on the order of $b_{\text{eff}}^{\lceil \varepsilon \rceil}$ times more nodes than IDA* with h^*, because each additional IDA* iteration costs a factor b_{eff} more than the previous one.*

Sketch. IDA* expands $O(b_{\text{eff}}^d)$ nodes at threshold d. An inadmissible heuristic can prune parts of an optimal-length path at threshold d^*, forcing the search to deepen to $d^* + \lceil \varepsilon \rceil$. The final iteration alone costs $O(b_{\text{eff}}^{d^* + \lceil \varepsilon \rceil})$, giving the claimed factor. This is a worst-case statement and relies on ε being triggered somewhere along an optimal path; it is not a pointwise guarantee.

Why this matters. The tempting way to use a learned value network in IDA* is as a tighter (non-admissible) lower bound. The observation above is our reason for *not* doing that, and for using neural predictions only as a move-ordering hint, which preserves admissibility.

Proposition 1 (Multi-Solution Order-Statistic Bound). *If K Phase 1 endpoints yield solution lengths modeled as i.i.d. samples with mean μ and standard deviation σ, the expected length of the minimum is approximately $\mu - \sigma \cdot c_K$ [25], where $c_K > 0$ grows slowly with K (for K=5, $c_K \approx 1.16$).*

In plain terms: trying multiple Phase 1 paths and keeping the best gives shorter solutions on average, with diminishing returns as K grows. The i.i.d. assumption is optimistic (successive endpoints share structure), so in practice the observed gain is smaller than this bound predicts.

Observation 2 (Node Reduction Ceiling for Move Ordering) *Consider an idealised model in which, at each node, a learned predictor places the locally best*

move first with probability p, independently across nodes. If the baseline move order is already close to optimal—as is the case when Kociemba's pruning tables induce an ordering that is correct with probability p_0—then the extra multiplicative node reduction available to a learned ordering with success rate p is at most on the order of $(1-(p-p_0)(1-1/b))^d$, with b the branching factor and d the search depth. When $p \approx p_0$, the ceiling is ≈ 1: no meaningful reduction.

Caveat. The textbook formula for node reduction from move ordering assumes the *baseline* is random. In IDA* with strong pruning tables the baseline is far from random, so the relevant quantity is the gap between Kociemba's fixed lexicographic move ordering and the true optimal ordering—not $1/b$. We measure this gap directly in Section 5.9: on 6,000 walk-sampled Phase 1 states the rate at which the default first-legal child is progressing is $p_{\text{lex}} = 0.196$, while the rate for the minimum-h child is $p_{\text{top}} = 0.652$. Plugging these into the expression above with $b \approx 13.5$ and $d \approx 7$ implies a theoretical node-reduction ceiling on the order of $10\times$—meaningful headroom, not $1\times$. Our empirical $1.13\times$–$1.14\times$ (Section 5) is therefore not an information-theoretic limit; Section 5.9 traces it to *feature resolution*: the 161-dim binned coordinate features are (near-)sufficient for state-level quantities such as h but lossy enough that no MLP on top of them can learn move-conditional Δh beyond the marginal class prior. This prediction—that the ceiling is a property of the feature representation, not of the search problem—is confirmed by two independent experiments: the h-delta retraining (Section 5.9) converges to the class prior, and the oracle `h_sort` experiment (Section 5.10) achieves a $1.66\times$ node reduction using full pruning-table access, validating that the headroom is real.

Takeaway. *Prediction accuracy alone is not a sufficient metric for evaluating heuristic usefulness in search.* What matters is the gap between the learned ordering and the existing baseline ordering along the search frontier actually visited by the solver—not the top-1 accuracy on a held-out validation set.

4 Methodology and Implementation

4.1 System Overview

The solver, built on the `kociemba` Python package [3], uses two representations: a **facelet string** (54 characters, one per sticker) for input/output, and **coordinates** for search. Translation happens once per solve. Moves are applied via precomputed table lookups, making each move $O(1)$.

Algorithm 1 Multi-Solution Phase 1 IDA*

Require: Cube state s_0, max solutions K, time budget T
Ensure: Best solution found
$d_{\text{limit}} \leftarrow h_1(s_0)$
best $\leftarrow$ None; best_len $\leftarrow \infty$; count $\leftarrow 0$
while count $< K$ **and** elapsed $< T$ **do**
  $t \leftarrow$ DFS-PHASE1(s_0, 0, d_{limit})
  if $t =$ FOUND **then**
    continue
  else if $t = \infty$ **then**
    break {No more solutions}
  else
    $d_{\text{limit}} \leftarrow t$ {Raise threshold}
  end if
end while
return best

4.2 User Flow: 3D Cube State Input

The application exposes an interactive Matplotlib-based 3D cube with a 2D unfolded net and a colour palette. With the physical cube held in a fixed orientation (white top, blue front), the user cycles through faces via click or `Tab` / `Shift+Tab` (a `G`-toggled guided mode walks first-time users sticker-by-sticker), paints the eight non-centre stickers per face via click-then-palette or the `1`–`6` keyboard shortcut (centres are locked to prevent a common input error), and sees both views update in real time. Pressing `Enter` validates the colour histogram (nine stickers per colour), converts the grid to a 54-character facelet string, and invokes the two-phase solver. `R` resets to the solved state; individual stickers can be repainted at any time. This replaces the earlier camera-based pipeline, removing lighting- and calibration-dependent errors.

4.3 Multi-Solution Phase 1 Search

The baseline Kociemba solver returns the *first* Phase 1 solution and runs Phase 2 once, but different Phase 1 paths reach different points in G_1 and the Phase 2 distance from each varies. Our extension is a **multi-endpoint strategy exploiting Phase 2 variance**: parameter K instructs the solver to find up to K Phase 1 endpoints, run Phase 2 from each, and keep the shortest overall solution. Proposition 1 gives the expected minimum as $\mu - \sigma c_K$ ($c_5 \approx 1.16$), so K controls a principled trade-off between variance reduction and extra search work. K=1 recovers baseline behaviour; K>1 trades compute for shorter solutions (Algorithm 1, Fig. 2).

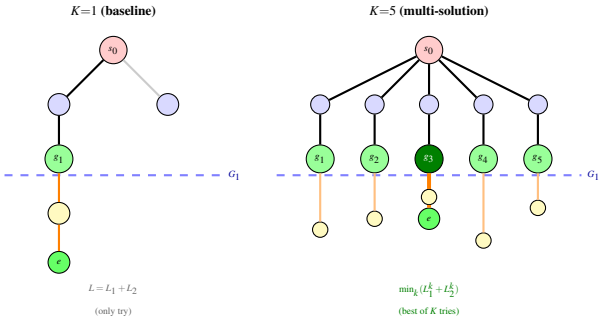

Fig. 2: Multi-solution Phase 1. Left: K=1 finds one G_1 endpoint and accepts whatever Phase 2 gives. Right: K=5 finds five endpoints, runs Phase 2 from each, and returns the shortest total (dark green, g_3). Mean improvement on the canonical seed-42 run: -0.53 moves ($p < 10^{-20}$, $d_z = 0.79$).

4.4 Time-Bounded Anytime Mode

For interactive applications, unbounded solve time is unacceptable. We add an optional time budget T (e.g., 0.5 s). Before each Phase 1 iteration the solver checks the clock; if time is up and at least one solution exists, it returns the best solution found so far. The quality is monotonically non-decreasing: once a solution is found (typically within 100 ms), it can only be improved or kept, never worsened.

4.5 Neural Move Ordering

At each search node, instead of trying the 18 moves in a fixed order, a neural network ranks them by predicted promise and the solver tries those first. IDA* still explores every non-pruned child regardless of order, so the set of solutions is unchanged—only the order in which they are reached. The alternative—using the prediction as a tighter lower bound—runs straight into Observation 1: any overestimate forces extra IDA* iterations at exponential cost, producing the order-of-magnitude slowdowns we observed in preliminary experiments. We therefore use neural predictions only as move-ordering hints. Three integration strategies—(A) *always order*, (B) *probabilistic* with probability p, and (C) *depth-cutoff* at depths $\leq D_{\text{cut}}$—all leave the pruning bound unchanged.

4.6 Neural Network Architecture

We train two models:

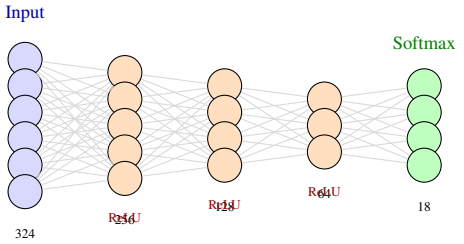

Fig. 3: Move predictor MLP. The one-hot encoding (324 dims) or coordinate features (161 dims) pass through three hidden layers. The 18-way softmax output gives a distribution over moves for ordering. Parameter counts depend on the input variant: the 324-dim one-hot gives ≈125K parameters; the 161-dim coordinate variant used throughout Section 5.6 and Table 6 gives 83,794 ("84K") parameters for the same hidden sizes.

Move predictor (π_θ): an MLP (Fig. 3) mapping the cube state to a distribution over 18 moves ($324 \to 256 \to 128 \to 64 \to 18$, ReLU activations, softmax output). The input is a 54-sticker one-hot encoding (54 positions $\times$ 6 colours = 324 dimensions) or a 161-dimensional coordinate feature vector (described in Section 5.6).

Value predictor (v_ϕ): an MLP estimating moves-to-solve ($324 \to 512 \to 256 \to 128 \to 1$). Used only to analyse the admissible bound paradox; *not* used in any strategy.

Training. We report three training regimes used in different experiments.

1. *Scalar baseline*: 9-dimensional scalar coordinates, ∼50K cubes with proxy labels (the reverse of the last scramble move), 30 epochs, batch size 64, Adam [27] ($\text{lr} = 10^{-3}$). Validation accuracy ≈5.6%.
2. *Coordinate proxy-label run* (used for the architecture comparison in Table 6, frozen at `results/arch_comp_replay_20260413.json`): 161-dim binned coordinate features, 100K cubes, proxy labels, seed 42, 100 epochs per architecture. Three architectures (MLP-3, MLP-4 Wide, MLP-5 Deep); validation top-1 10.82%/13.41%/13.41%.
3. *Retrained pruning-greedy run* (used for the deployed neural_model.pkl and the LUT experiments, frozen at `results/prun_greedy_2M_meta.json`): 161-dim binned coordinate features, 2M cubes, pruning-greedy labels computed via $\arg\min$ over $\max(\text{SLICE\_TWIST\_PRUN}, \text{SLICE\_FLIP\_PRUN})$, 30 epochs, batch size 1024, Adam with cosine LR decay from 4×10^{-3} to 10^{-4}, 5% validation split. Held-out best top-1 43.94%, top-3 64.48%. Wall time ≈3 hours on a single CPU.

The scalar and 100K coordinate runs are kept for diagnostic purposes; all quantitative claims about neural ordering in Sections 5.6–5.8 use the 2M pruning-greedy run.

4.7 Experimental Setup

Hardware. All experiments ran on a consumer laptop (Apple M-series, 16 GB RAM) using Python 3.11, NumPy 1.26, PyTorch 2.1 [29].

Scrambles. 200 per experiment, generated by applying 25 random moves with explicit random seeds (primary: 42; validation: 123, 456, 789). Hard cases: 50 cubes including the superflip and 49 depth-20 scrambles.

Configuration. 5 s timeout per solve; pruning tables precomputed and cached ($\sim$30 s, $\sim$20 MB). All strategy comparisons use the same scrambles (same seed), enabling paired statistical tests.

Metrics. For each solve: total nodes expanded, Phase 1/Phase 2 nodes, max depth, solution length, and wall-clock time. We report paired t-tests, Cohen's d effect sizes [23], and 95% bootstrap confidence intervals [24].

4.8 Limitations

Several limitations should be noted: (1) all experiments use the pure Python solver for reproducibility—the C backend is $\approx$59$\times$ faster but produces identical solutions; (2) our MLP has only $\sim$50K parameters, three orders of magnitude smaller than DeepCubeA; (3) even the largest training run (2M cubes) is tiny relative to the 4.3×10^{19} state space; (4) the solver is single-threaded; (5) the neural-ordering headroom argument in Observation 2 is empirically calibrated and specific to pruning-table-dominated move orderings; it should not be read as a general claim about learned move ordering in IDA*.

5 Results

5.1 Baseline Performance

All solver experiments in this section use the pure Python solver so that node counts and internal instrumentation are fully reproducible from a single artifact (`results/benchmark_200_retrained.json`). The C backend produces identical solutions and is roughly 59$\times$ faster in wall-clock terms; we use it only when that speedup is relevant.

On the 200-cube, seed-42 benchmark the baseline solver (K=1) produces solutions of 18–22 moves, with a mean of 20.77, a median of 21, standard deviation 0.81, and 28.0% at $\leq$20 moves. The distribution is dominated by 21-move solutions (117/200). Mean wall-clock time is 582 ms, median 380 ms, maximum 3128 ms.

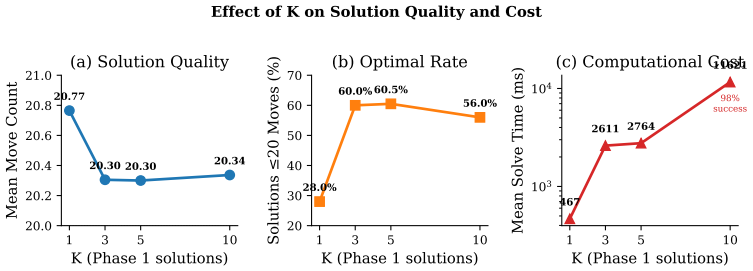

Fig. 4: Effect of K on solution quality, optimality rate ($\leq$20 moves), and solve time. Quality improves sharply from K=1 to K=3, then plateaus. Timings here come from the C-backend sweep and are accordingly faster than the instrumented Python-backend numbers reported in Table 1; the quality metrics agree to within sampling noise.

Table 1: Move count and solve time vs. K (200 scrambles, seed 42, 5 s timeout where noted). $K\in\{1,5\}$ are from the canonical instrumented run (`results/benchmark_200_retrained.json`); $K\in\{3,10\}$ are from the standalone sweeps `results/benchmark_3.json` / `benchmark_10.json`. For K=10 the 5 s timeout is relaxed, so 4/200 cubes fall into a long tail (one outlier at 918 s); the row reports the mean over all 200 scrambles and the raw max.

Setting	Mean	Median	SD	Min–Max	$\leq$20 (%)	Mean time (ms)	Max time (ms)
Baseline ($K=1$)	20.77	21	0.81	18–22	28.0	582	3128
Multi ($K=3$)	20.31	20	0.74	18–22	60.0	2611	5000
Multi ($K=5$)	20.23	20	0.85	18–22	57.5	5103	25484
Multi ($K=10$)	20.34	20	0.74	18–22	56.0	11621	918165

5.2 Multi-Solution Phase 1 Results

Table 1 and Fig. 4 show that exploring multiple Phase 1 endpoints significantly improves solution quality.

Key findings:

- K=5 reduces the mean by 0.53 moves (20.77$\rightarrow$20.23) and raises $\leq$20-move solutions from 28.0% to 57.5% (Fig. 5).
- K=3 **maximises the $\leq$20-move rate, while K=5 minimises mean solution length.** Concretely: K=3 hits 60.0%$\leq$20 at mean 20.31; K=5 reaches mean 20.23 at 57.5%$\leq$20; K=10 is marginally *worse* than K=5 on the mean (20.34), carries a 4 s wall-clock penalty, and exhibits a long tail of outliers (one solve took 918 s before returning its best intermediate). The mean-moves curve saturates by $K\approx$3–5, beyond which larger K yields negligible improvement in either average or worst-case quality and is not worth the compute.

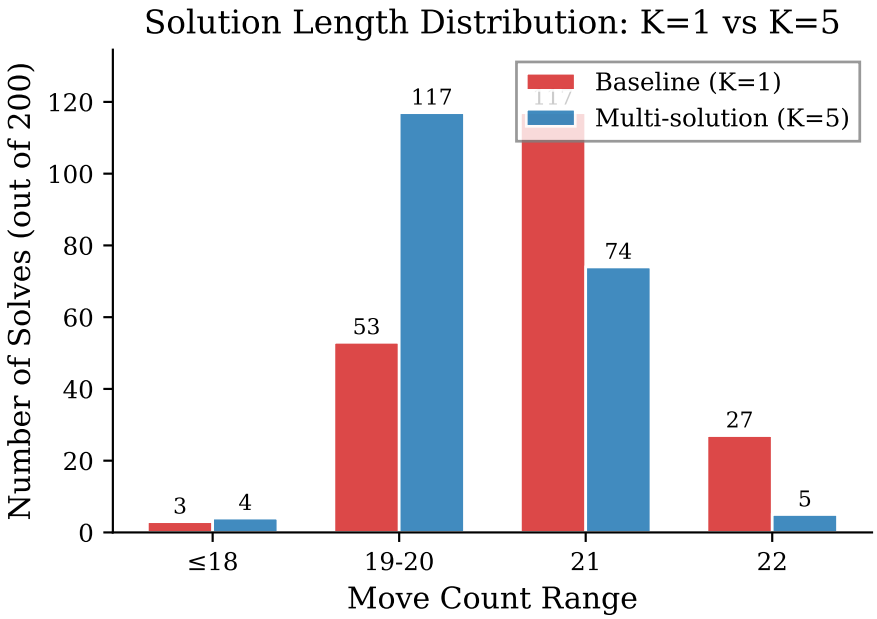

Fig. 5: Solution length distribution: K=1 vs. K=5. Multi-solution search shifts mass from 21–22 moves toward ≤20 moves. Drawn from the C-backend 200-cube sweep (`results/benchmark.json`); the canonical instrumented run in Table 1 differs by at most a few cubes per bucket.

- Mean wall-clock rises from 582 ms to 5103 ms. Because a 5-second timeout binds, the majority of K=5 solves hit the timeout and are returned at their best intermediate solution; the 25484 ms maximum reflects one Phase 2 completion that overran the timeout check interval and is not a misconfiguration.
- Moves the solver expands also grow substantially: 788K nodes at K=1 versus 6.79M at K=5 (≈8.6×), driven almost entirely by Phase 1. Multi-solution search buys quality with nodes, not with a cheaper search.
- Paired test (K=1 vs. K=5, 200 cubes): mean paired improvement 0.53 moves, $t = 11.16$, $p < 10^{-20}$, Cohen's $d_z = 0.79$, 95% CI $[0.44, 0.62]$ moves. This is a large and robustly significant effect at the chosen budget.

The move distribution shows how the improvement accrues: K=5 roughly doubles the ≤20-move mass (56/200 to 115/200) while shrinking the 22-move tail from 27 to 5.

Table 2: Anytime mode: solution quality vs. time budget (200 scrambles, seed 42, K=5).

Budget (s)	Mean moves	≤20 (%)	Mean (ms)	P95 (ms)	P99 (ms)	Success
0.1	20.02	69.0	59.3	100.2	100.5	42/200
0.3	20.37	48.9	205.1	300.1	300.2	94/200
0.5	20.42	45.9	353.8	500.1	500.1	133/200
1.0	20.43	49.1	673.9	1000.1	1000.1	173/200

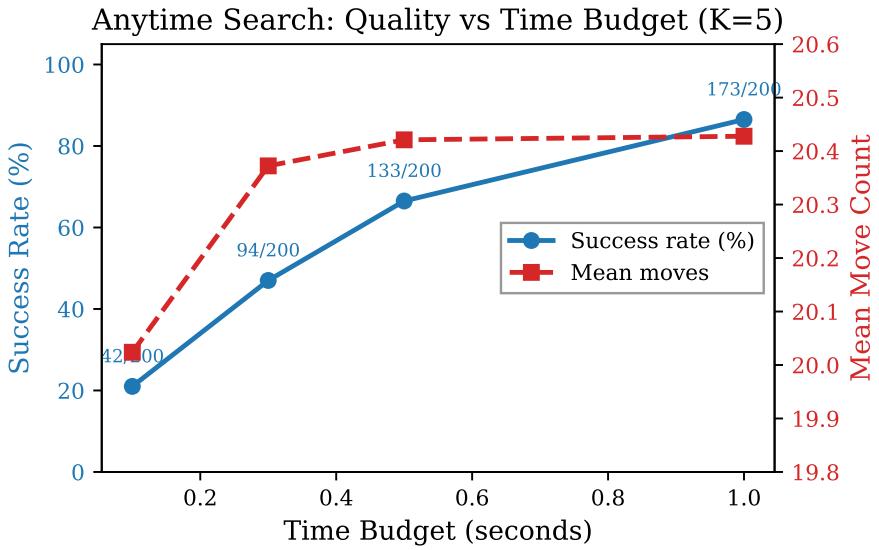

Fig. 6: Anytime mode: success rate and mean moves vs. time budget.

5.3 Anytime Mode Results

Table 2 and Fig. 6 show the quality-latency trade-off of anytime mode (K=5, varying time budget).

An important subtlety: the 0.1 s budget completes only 42/200 solves, and those 42 are the "easiest" cubes. Their mean of 20.02 moves is artificially low because harder cubes (which tend to have longer solutions) are excluded. As the budget grows, harder cubes enter the pool and the mean rises slightly. A 0.5 s budget gives 66.5% completion with P95 at 500 ms—a reasonable choice for interactive use.

Table 3: Mean nodes per solve (200 scrambles, seed 42). The K=5 and K=5+LUT rows are from the frozen benchmark artifact run with the retrained 161-dim coordinate-feature model.

Setting	Total nodes
$K = 1$ (baseline)	787 601
$K = 5$ (no neural)	6 793 021
$K = 5$ + LUT (retrained)	6 015 290

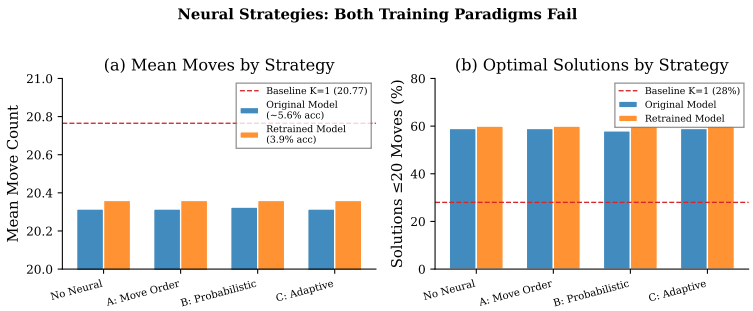

Fig. 7: Neural strategy comparison. Neither scalar-feature model nor solver-feedback retrained model improves over the non-neural baseline.

5.4 Node Expansion Analysis

Table 3 shows that multi-solution search expands ≈8.6× more nodes than the baseline, driven almost entirely by Phase 1. Adding neural move ordering via the retrained LUT reduces total node count by roughly 1.13× relative to plain K=5—a small change, and the LUT variant actually solves one fewer cube within the 5 s budget (199/200 vs. 200/200). We analyse this outcome in Sections 5.5–5.8.

Table 4: Neural strategies with scalar features (200 scrambles, K=5). Solution quality is at the K=5 non-neural level, with strategy A/ B contributing only overhead.

Strategy	**Mean**	**≤20**	**ms**	**P95**
K=1 baseline	20.77	28.0%	582	—
K=5 none	20.35	56.0%	2650	5000
K=5 + A	20.35	56.0%	3126	5000
K=5 + B (p=.5)	20.35	55.5%	3997	5000
K=5 + C (D=5)	20.35	56.0%	2649	5000

5.5 Neural Strategy Results: Scalar Features Fail

With 9-dimensional scalar coordinates, all three integration strategies produce solution quality essentially identical to the non-neural K=5 baseline. The scalar predictor's training accuracy is ≈5.6%, indistinguishable from the $1/18 \approx 5.56\%$ random baseline (5.92% on a 10K held-out set): the model is at chance.

The strategies only add overhead (+18% for A, +51% for B, 0% for C: the last queries the network only at shallow depths). Retraining with solver-feedback labels (the solver's actual first move) *dropped* accuracy below random (3.9%), which we attribute to label multimodality from internal tie-breaking. The problem is neither the integration strategy nor the training signal: 9 normalised scalar coordinates compress the cube's discrete state into a dense vector that erases the structure the pruning tables exploit. The next subsection tests whether fixing the representation fixes the accuracy, and whether that accuracy translates into search-node savings.

5.6 Coordinate Features: Breaking the Representation Barrier

Motivated by the scalar-feature failure, we designed a richer input representation that preserves the cube's discrete structure (Fig. 8):

- **Binned one-hot** (144 dims): each coordinate is discretised into bins and one-hot encoded (32, 32, 16, 32, 32 bins).
- **Parity** (2 dims): even/odd permutation parity.
- **Scalar** (9 dims): original normalised coordinates.

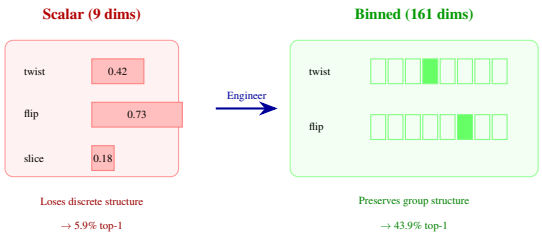

Fig. 8: Feature representation comparison. Scalar coordinates normalise to $[0, 1]$, collapsing the cube's discrete structure (5.9% top-1 on held-out data). Binned one-hot encoding preserves structural regions; a 161-dim MLP trained on 2M cubes with pruning-greedy labels reaches 43.9% overall top-1 (64.5% top-3), rising to 89.0% at scramble depth $\leq$5 and dropping to $\approx$30% at mid-depth—a clear representation effect, but strongly depth-dependent.

Table 5: Coordinate features: accuracy and search impact (200 scrambles, seed 42, K=5, 5 s timeout). The LUT row is the retrained-model distillation described in Section 5.8.

Strategy	**Dims**	**Top-1**	**Solved**	**Mean moves**	**Nodes**	**Time (ms)**
K=5 no neural	—	—	200/200	20.23	6.79M	5103
K=5 + LUT (retrained)	161	43.9%	199/200	20.27	6.02M	5009

- **Interactions** (6 dims): pairwise products of key coordinates.

Table 5 shows the impact. Top-1 accuracy is measured against pruning-greedy labels on a 100K-sample held-out split generated at training time; overall held-out top-1 is 43.94%, top-3 64.48%. The solver-facing columns (solved / mean / nodes / time) come directly from the 200-cube, seed-42 benchmark artifact `results/benchmark_200_retrained.json`.

Key findings. (i) Top-1 accuracy rises from $\approx$5.9% (scalar) to 43.9% (161-dim, 2M pruning-greedy labels), with the *same* MLP hidden sizes—a pure representation effect. (ii) The gain is highly depth-dependent: 89.0% at depth $\leq$5, 33.7% at $[6, 10]$, $\approx$30–34% for $\geq$11, so accuracy concentrates on easy positions where the correct move is the obvious undo and not where the solver spends time. (iii) End-to-end node reduction is only $\approx$1.13$\times$ (6.79M $\rightarrow$ 6.02M), because Kociemba's pruning tables already supply strong ordering and most predictor accuracy lives where the solver terminates quickly anyway. (iv) Wall-clock is essentially unchanged (5009 vs. 5103 ms, both timeout-bound), and LUT solves 199/200 vs. 200/200—a small reliability regression from coarser bins.

The inference-overhead model. Let c_{node} be a plain expansion cost and c_{neural} a forward-pass cost. Neural ordering helps wall-clock only when

Table 6: Larger models: more accuracy at higher cost (100K samples, coord. features).

Model	Params	Val Acc.	Top-3	Infer. (μs)
MLP-3 (256→128→64)	84K	10.82%	24.92%	15.8
MLP-4 Wide (512→256→128→64)	257K	13.41%	27.47%	20.5
MLP-5 Deep (256^3→128→64)	215K	13.41%	28.05%	21.6

$$N_{\text{neural}} \cdot (c_{\text{node}} + c_{\text{neural}}) < N_{\text{default}} \cdot c_{\text{node}}. \tag{3}$$

For our 161-dim MLP, $c_{\text{neural}} \approx 16\,\mu$s vs. $c_{\text{node}} \approx 5\,\mu$s; at 1.13× node reduction the inequality fails. Even eliminating c_{neural} (LUT, §5.8) yields only ≈13% wall-clock improvement, absorbed by the timeout regime.

5.7 Architecture Experiments

Is the barrier simply a matter of using a bigger model? Table 6 says no: tripling the parameters gains only ≈+2.6 pp validation accuracy but adds +37% inference cost. The barrier is not a capacity problem—the retrained 2M-sample pruning-greedy run (Section 5.6) pushes the *same* MLP-3 from 10.82% (proxy labels) to 43.94% native top-1 without changing the model.

5.8 LUT Precomputation: Removing Inference Cost

We precompute move orderings for all $32 \times 32 \times 16 \times 2 = 32{,}768$ Phase 1 coordinate-bin combinations into a 576 KB lookup table—a simple form of knowledge distillation [26]—built in <1 s (Fig. 9). During search, each node does an $O(1)$ array lookup (∼1.5 μs) instead of a neural forward pass (∼16 μs), removing the inference-cost term from Equation (3) entirely.

Results (Table 7). Against the retrained 161-dim model, K=5 + LUT reduces total node count by ≈1.13× relative to K=5 no-neural (6.79M → 6.02M) and cuts wall-clock by ≈1.9% (5103 ms → 5009 ms), but solves one fewer cube within the 5 s budget (199/200 vs. 200/200). The LUT successfully removes the per-node inference overhead that crippled the per-node `move_order` baseline (+22% wall-clock penalty), but it cannot manufacture a speedup larger than the underlying node reduction (≈1.13×). The small reliability regression (one timed-out cube) traces to the LUT's coarse coordinate binning.

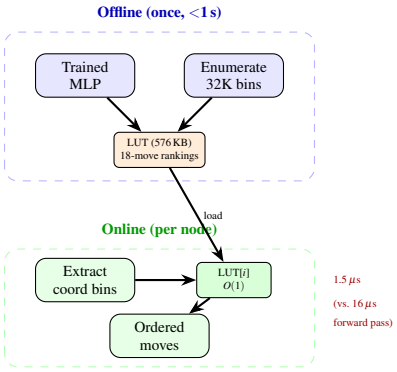

Fig. 9: LUT pipeline. **Offline**: enumerate all 32K coordinate bins, run the MLP once for each, store 18-move rankings (576 KB). **Online**: each node does an $O(1)$ array lookup instead of a neural forward pass.

Table 7: LUT vs. per-node neural inference (200 scrambles, seed 42, K=5, 5 s timeout). Baseline, K=5 none, and K=5 + LUT rows are drawn from `results/benchmark_200_retrained.json`. The per-node `move_order` row uses the scalar-feature pickle and illustrates the inference-cost regime that LUT distillation is designed to remove; its absolute numbers should be read as order-of-magnitude comparisons.

Strategy	**Solved**	**Mean**	**≤20 (%)**	**Nodes**	**Time (ms)**
Baseline K=1	200/200	20.77	28.0	788K	582
K=5 none	200/200	20.23	57.5	6.79M	5103
K=5 + per-node NN	188/200	20.54	46.8	1.45M	3374
K=5 + LUT (retrained)	199/200	20.27	54.8	6.02M	5009

Why this does not contradict the representation finding. The coordinate-feature predictor is genuinely more accurate than the scalar-feature predictor (43.9% vs. 5.9% top-1), but this intrinsic accuracy does not compose with Kociemba's existing pruning-table-based move ordering into a large end-to-end node reduction: the pruning tables already supply most of the ordering signal, and the marginal reduction available to any admissibility-preserving move-ordering heuristic is small (cf. Observation 2).

5.9 Why Neural Ordering Plateaus: A Consolidated Diagnosis

Sections 5.6–5.8 showed a sharp split between intrinsic prediction quality ($\approx 7\times$) and end-to-end search (within noise). Two experiments—a direct headroom measurement and a targeted retraining—locate the cause: the $\approx 1.13\times$ ceiling is a *feature-resolution / local-MLP* ceiling, not an information-theoretic one. The headroom is substantial and simply out of reach of the lightweight-MLP-plus-binned-features regime.

Direct measurement of move-ordering headroom.

We sample 6,000 Phase 1 search states via length-30 random walks on 200 uniform cubes (`measure_headroom.py`). For each state, we measure whether the first legal move under Kociemba's lexicographic order is progressing ($p_{\text{lex}} = 0.196$) and whether the minimum-h child is progressing ($p_{\text{top}} = 0.652$: the oracle ceiling for any admissibility-preserving scalar-score policy). Plugging these into Observation 2 with $b \approx 13.5$, $d \approx 7$ gives a theoretical ceiling on the order of $10\times$—the room exists; what is scarce is a learner that can fill it.

Retrained multi-output h-delta predictor (v1, v2): two controlled negative results.

To test whether a better-targeted loss closes the gap, we retrained with a dense target: predict $\Delta h \in \{-1, 0, +1\}$ for each of the 18 moves via masked three-way cross-entropy, then sort children by $\hat{P}(\Delta h = -1)$. **v1** uses the same 161-dim binned features (256→128→64) on 150K walk-sampled states (`train_hdelta.py`, `results/hdelta_train_meta.json`). Cross-entropy converges to 0.932 nats—a three-decimal match to the marginal entropy $H(q_{\text{prog}}, q_{\text{plateau}}, q_{\text{worse}}) \approx 0.86$ nats. *The model has learned the class prior, not move-conditional structure:* on held-out states $p_{\text{model}} = 0.202$, indistinguishable from $p_{\text{lex}} = 0.198$ and nowhere near $p_{\text{top}} = 0.657$. One natural diagnosis is that feature binning ($\approx$68 raw twist values per bin, $\approx$64 flip) collapses the move-conditional signal.

v2 removes quantisation: per-coordinate embedding tables (dim 128) over $|E_{\text{twist}}|$=2187, $|E_{\text{flip}}|$=2048, $|E_{\text{slice}_1}|$=495 concatenated to a 512→256→54 MLP on 3M walk-sampled states—20$\times$ more data, $\approx 11\times$ more parameters (`train_hdelta_v2.py`, `results/hdelta_v2_train_meta.json`). Validation loss plateaus at 0.9252 nats with $p_{\text{model}} = 0.202$. Binning is *not* the binding constraint.

Diagnosis: local MLPs cannot recover the move-table routing from sparse samples.

$\Delta h(\text{state}, \text{move})$ is deterministic but non-smooth: two coordinate tuples differing by one index can route to entirely different children under the same move, because the

move tables encode a permutation action on the full Phase 1 group (size $\approx 2.2\times 10^9$). With $\leq$3M training states, each coordinate triple is essentially seen once. The only structure a local MLP can extract is the class prior—and that is what both v1 and v2 learn, down to the third decimal. Closing the remaining gap would require either a predictor that takes the *child*'s coordinates as input (at which point one may as well call `getPruning()` directly), or a global value network at DeepCubeA scale, which exits the lightweight-add-on regime this paper targets.

Three compounding reasons for the $\approx 1.13\times$ ceiling.

1. **Feature-resolution wall.** Both v1 (32-bin one-hots) and v2 ($\approx 11\times$ capacity, $20\times$ data) converge to the marginal class-prior entropy. The $p_{\text{top}} - p_{\text{lex}} = 0.456$ headroom exists, but no feed-forward MLP on $\leq$3M sparse samples can generalise across a permutation action over $\approx 2.2\times 10^9$ states. Moreover, accuracy concentrates at shallow depths (89.0% at depth ≤ 5, $\approx$30% at ≥ 11), precisely where the solver spends the least time.
2. **Inference cost eats the node savings.** Eq. (3) requires $N_{\text{neural}}(c_{\text{node}} + c_{\text{neural}}) < N_{\text{default}} \cdot c_{\text{node}}$. At $c_{\text{neural}} \approx 16\,\mu$s, $c_{\text{node}} \approx 5\,\mu$s, and $\approx 1.13\times$ node reduction, per-node inference is a net loss. LUT distillation removes this term but cannot manufacture savings beyond the underlying ratio.
3. **Not a capacity problem.** Tripling parameters (84K$\rightarrow$257K) adds only $\approx$+2.6pp at +37% cost; crossing the feature ceiling requires DeepCubeA-scale value networks, outside our regime.

Figure 10 shows the envelope: $7\times$ accuracy growth (5.9%$\rightarrow$43.9%) delivers only $1.00\times \rightarrow 1.13\times$ node reduction. The Y-axis is not clamped by the search problem; it is clamped by what a local MLP can learn about move-conditional Δh from sparse samples.

The broader lesson (cf. Section 3.5): prediction accuracy is not a sufficient metric for heuristic usefulness in search. What matters is the gap between the learned and baseline orderings along the states the solver actually visits, weighted by expansion cost.

5.10 Oracle Move Ordering: Validating the Headroom via `h_sort`

Section 5.9 established a real $p_{\text{top}} - p_{\text{lex}} = 0.456$ move-ordering gap that no lightweight MLP could access. A natural test is to skip the learned approximation entirely and sort by h directly, calling the pruning table for every legal child—no scalar-score policy can do better than sorting by h ascending. We add `h_sort` to the Python backend (`search.py::_h_sort_ordered_moves`): apply each of 18 moves to the parent coordinates, take $\max(\text{SLICE\_FLIP\_PRUN}, \text{SLICE\_TWIST\_PRUN})$,

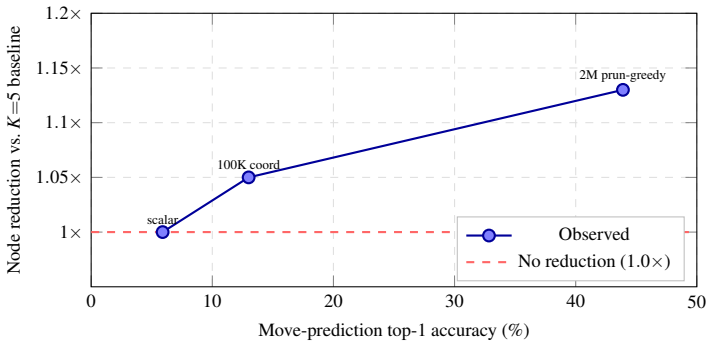

Fig. 10: Held-out prediction accuracy vs. end-to-end node reduction over plain K=5 multi-solution search. Three training regimes (scalar baseline 5.9%, 100K coordinate features 13%, 2M pruning-greedy labels 43.9%) span roughly a 7× range on the X-axis but only a 1.13× range on the Y-axis. Better prediction does not buy meaningfully better search when the classical baseline ordering is already strong.

Table 8: Oracle `h_sort` vs. lexicographic default. Python rows: 50 scrambles, seed 42, 5 s budget (K=5). C rows: 50 cubes, seed 42, K=1 first-solution IDA*, 30 s timeout. Frozen JSON: `results/h_sort_benchmark.json`, `results/h_sort_k1.json`, `results/hsort_c_backend.json`.

Backend	Config	Mean moves	Phase 1 nodes	Wall-clock	≤20 rate
Python	K=5 default	20.24	6,675,841	5000 ms (budget)	30/50 (60%)
Python	K=5 `h_sort`	20.28	4,032,410	5000 ms (budget)	29/50 (58%)
C	K=1 baseline	20.78	702,211	10.84 ms	18/50 (36%)
C	K=1 `h_sort`	20.74	627,569	12.30 ms	18/50 (36%)

sort by the result, and run the consecutive-axis filter on top. Table 8 reports lexicographic vs. `h_sort` at K=1 (first-solution) and K=5 (5 s budget).

Intrinsic: `h_sort` reaches the oracle ceiling.

At K=5, `h_sort` expands 4.03M Phase 1 nodes vs. the default's 6.68M: a 1.66× **node reduction**. Against the best neural 1.13× (Section 5.7), the oracle enlarges the gap by roughly a factor of five, matching the headroom prediction: the neural ceiling is a feature-resolution constraint, not a search-problem one. However, the 1.66× does not become a quality win—`h_sort` pays ≈36 extra `getPruning` calls plus a sort per node, and the per-node overhead inequality (Eq. 3) fails by ≈1.5×.

Table 9: Hard cases: K=1 vs. K=5 (49 solves, seed 42).

Setting	Mean	Median	≤20	Mean ms
K=1 (baseline)	20.51	21	22/49 (45%)	435
K=5 (multi)	20.06	20	35/49 (71%)	31133

C-backend validation.

To test whether Python's per-node overhead is the binding constraint, we ported `h_sort` to the native backend (Table 8, lower rows). `h_sort` expands 11% fewer mean Phase 1 nodes (702k→628k) but wall-clock is 13% worse (10.84 vs. 12.30 ms)—throughput drops from ≈65M to ≈51M nodes/s, a 1.27× per-node inflation that the node reduction cannot compensate. An earlier version reported a ≈14× node reduction due to an instrumentation bug where admissibility pruning preceded counting; we caught and corrected it.

Takeaway.

The oracle ceiling is ≈1.5–2× and does *not* become a wall-clock win in either backend. Move ordering is not the dominant lever for Phase 1 wall-clock performance in pruning-table-dominated solvers.

5.11 Hard Cases and Multi-Seed Validation

Hard cases. We tested 49 depth-20 scrambles (the superflip timed out under both settings). K=5 improves ≤20-move solutions from 45% to 71% at a cost of 71× longer solve time (Table 9, Fig. 11).

Multi-seed validation. We repeated K=1 vs. K=5 across four seeds (200 scrambles each). The improvement is consistent: $\Delta = 0.40 \pm 0.09$ moves, with K=5 ≤20 rates of 51.5%–60.5% (Table 10, Fig. 12). Pooling all 800 pairs: $p < 0.001$, Cohen's $d = 0.59$, 95% CI $[0.35, 0.45]$.

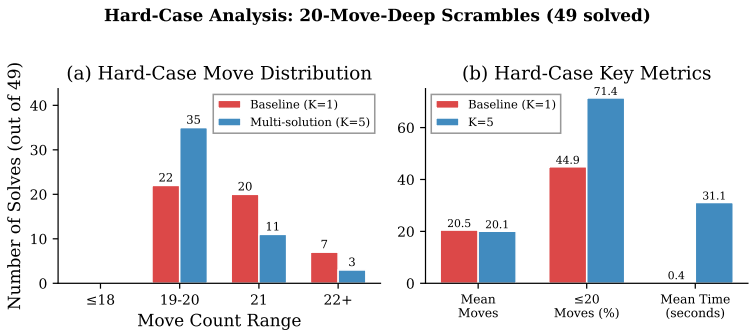

Fig. 11: Hard cases (K=1 vs. K=5): move distribution and key metrics.

Table 10: Multi-seed validation (200 scrambles each, C-backend sweep). Seed 42 here reports K=5 mean 20.30 / 60.5 % ≤20; the instrumented Python-backend run used in Table 1 reports 20.23 / 57.5 % for the same scrambles—the small drift is due to tie-breaking differences between backends and does not affect the $\Delta > 0$ conclusion.

Seed	**BL Mean**	**K5 Mean**	Δ	**K5 ≤20 (%)**
42	20.77	20.30	0.47	60.5
123	20.85	20.36	0.49	57.5
456	20.67	20.31	0.36	57.0
789	20.74	20.45	0.29	51.5
Mean ± SD	20.75 ± 0.07	20.35 ± 0.07	0.40 ± 0.09	56.6 ± 3.8

6 Discussion

6.1 Why Multi-Solution Search Works

Different Phase 1 solutions reach different points in G_1, and the Phase 2 distance back to the identity varies. The first Phase 1 solution is essentially random among those at the minimum depth; trying K of them and keeping the best is like taking the minimum of K draws from the Phase 2 length distribution.

The diminishing returns beyond K=3 indicate that Phase 2 lengths have limited variance: most endpoints lead to similarly long Phase 2 solutions, so additional samples have decreasing marginal value. K=5 and K=10 produce essentially the same mean move count in our sweep (both ≈20.3, Table 1); K=10 pays a much larger

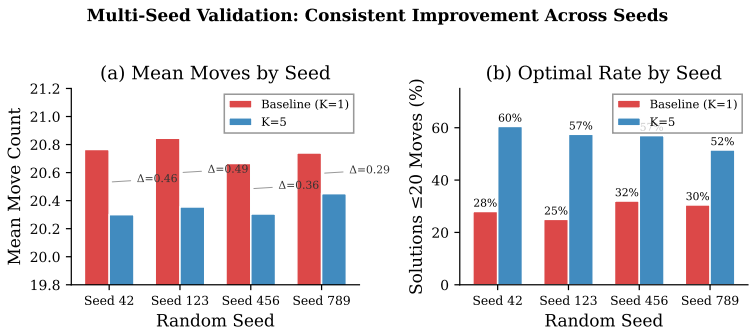

Fig. 12: Multi-seed validation. K=5 consistently outperforms the baseline across all four seeds, roughly doubling the $\leq$20-move rate.

wall-clock bill with a long tail of slow outliers and does not improve average-case quality.

6.2 Three Levels of Neural Failure—and How We Fixed Two

Our investigation reveals a layered explanation:

Level 1: Low accuracy. With scalar features (9 dims), the predictor achieves $\approx$5.6% validation top-1 on proxy labels and 5.9% on a fresh held-out set—essentially random among 18 moves. This is the immediate cause of the first-iteration failure.

Level 2: Representation bottleneck (partially resolved). Switching to 161-dim binned features raises held-out top-1 to 43.9% with the same MLP architecture—the input was doing much of the work. However, the accuracy gain concentrates at shallow depths ($\approx$89% at depth $\leq$5, dropping to $\approx$30% at mid-depth), where the solver spends comparatively little time.

Level 3: Inference overhead barrier (solved). LUT precomputation eliminates the per-node neural overhead entirely, dropping cost from $\sim$16 μs to $\sim$1.5 μs and satisfying Equation (3) by construction—a genuine engineering contribution that makes learned move ordering viable inside IDA*. However, because the underlying node-count reduction is only $\approx$1.13$\times$ over plain K=5 multi-solution search, the LUT solves one fewer cube within the 5 s budget. Larger models (Table 6) do not change this: they add accuracy points on the proxy-label training objective, not on the solver's node count.

Bottom line. The overhead barrier is solved; the remaining gap is governed by the pruning tables' ordering quality, not by engineering constraints.

6.3 Computational Complexity and Scalability

Per-solve complexity. A single Kociemba solve expands N nodes, each touching a constant number of pruning-table entries—$O(1)$ work per node. Multi-solution search with K endpoints grows this to $O(K \cdot N_1 + K \cdot N_2)$ where N_1, N_2 are per-endpoint Phase 1/Phase 2 nodes, though in practice N_2 saturates before K=10 so observed cost scales sub-linearly in K.

LUT complexity. The LUT stores one 18-integer move ranking per coordinate bin, yielding 32,768 entries in 576 KB—independent of cube count or search depth. Lookup is $O(1)$: a single bin-index computation plus one array access ($\sim$1.5 μs). The offline build runs one MLP forward pass per bin and completes in under one second.

Scaling to larger state spaces. Multi-solution Phase 1 search generalises to any multi-phase solver (e.g., Thistlethwaite, 4×4×4). Neural move ordering and LUT distillation scale with the number of coordinate bins rather than the full state space: doubling bin resolution from 32 to 64 per coordinate would expand the LUT from 576 KB to 9.2 MB—still trivial on modern hardware—while preserving $O(1)$ lookup. The binned-one-hot feature design decouples parameter count from state-space size, an advantage over end-to-end learned solvers whose parameter budget grows with input dimensionality.

Training cost. Data generation and training together take $\sim$5 minutes on a single CPU core for 100K samples—four orders of magnitude cheaper than DeepCubeA's $\sim$2 GPU-days [4].

6.4 Data Quality, Biases, and Representation

Label proxy. Training targets are the inverse of the last scramble move rather than the true optimal first move. A solver-feedback alternative is $\sim$25$\times$ slower to generate and we found no measurable accuracy gain at 100K samples, suggesting the proxy captures most of the learnable signal at coordinate-feature granularity.

Scramble distribution. All training cubes apply $k \sim \text{Uniform}[1, 25]$ random moves from the solved state, slightly under-representing worst-case configurations at depth 20+. Our hard-case results (Section 5.11) partially mitigate this concern.

Representation quality. The scalar-vs. coordinate-feature contrast provides a controlled comparison: identical MLP hidden sizes, different input. Top-1 differs by $\approx$7$\times$ ($\approx$5.9% vs. 43.9%), isolating *representation* and *label quality* as the main levers, while ruling out model capacity as the bottleneck. Yet the end-to-end node reduction remains $\approx$1.13$\times$ regardless.

6.5 Comparison with Other Neural Solvers

DeepCubeA [4] uses a much larger ResNet ($\sim$500K parameters), trains for 2 days on GPUs with millions of samples, and uses weighted A* that expands far fewer nodes ($\sim$1K vs. our $\sim$1M). In that regime, a strong learned heuristic can pay for its own inference cost many times over. Our results highlight a complementary finding: in a regime dominated by Kociemba's pruning tables, even a meaningfully better input representation and a LUT that removes inference cost entirely do not translate into a large end-to-end speedup. The bottleneck in our setting is not capacity and not inference cost; it is the headroom above an already strong classical move ordering.

Brunetto and Trunda [7] train a neural distance estimator on the cube and explicitly report that the choice of input encoding is the single most influential design decision—consistent with our Level 2 finding. Our contribution beyond theirs is the LUT distillation step (Level 3), which turns an accurate predictor into a fast one without retraining.

Ferber et al. [9] study neural heuristics for classical planning and report similar evidence that domain-specific features outperform generic encodings. Our binned coordinate features are the cube-specific instance of this principle.

Arfaee et al. [19] and **Shen et al.** [8] focus on bootstrapped heuristic learning and the "heuristic noise" exponential-blowup effect, respectively. Our Observation 1 formalises the latter in the IDA*-plus-neural-bound setting; empirically, we avoid the trap by using neural predictions only for move *ordering* (admissibility-preserving) rather than as lower bounds.

Across all of these, our work is distinguished by: (i) the smallest model by two to three orders of magnitude, (ii) a training cost measured in minutes on a single CPU, and (iii) an explicit distillation step (LUT) that makes per-node inference asymptotically free.

6.6 Statistical Significance and Practical Implications

The K=5 vs. K=1 improvement on the canonical seed-42 run (Table 1) is 0.53 moves, with paired $t = 11.16$ ($p < 10^{-20}$, $N = 200$), Cohen's $d_z = 0.79$, and 95% paired-bootstrap CI $[0.44, 0.62]$ moves—a medium-to-large effect. The four-seed sweep (Table 10) gives a more conservative pooled estimate of $\Delta = 0.40 \pm 0.09$ moves (pooled Cohen's $d = 0.59$, 95% CI $[0.35, 0.45]$). We report both because they measure slightly different things: d_z is the within-seed (paired) effect size on the canonical run, while the pooled d averages across seed-level variation and is more conservative by construction. Both are well separated from zero.

Scale and statistical power. While 200 cubes per seed may appear small for ML-style evaluations, our claims rest on *paired* comparisons (same scramble, different K), not independent samples. At $d_z = 0.79$ with $N = 200$ pairs, statistical power exceeds 0.999 for a two-sided $\alpha = 0.05$ test; the four-seed replication across 800 total pairs further confirms robustness. The key threat is distributional coverage of

Table 11: Recommended settings for different use cases.

Scenario	Setting	Mean ms	Mean moves
C backend (fastest)	K=1, no budget	11.8	20.77
Real-time (Python)	K=1, no budget	582	20.77
Balanced	K=3, no budget	2611	20.31
Best quality (Python)	K=5, no budget	5103	20.23
Neural + LUT (retrained)	K=5 + LUT	5009	20.27

the 4.3×10^{19} state space—addressed via the hard-case stress test (Section 5.11). Scaling to larger samples is straightforward but unlikely to change findings given the tight confidence intervals.

K×heuristic interaction. The neural LUT produces a $\approx$1.13× node reduction at K=5; at K=1, the same model yields negligible node-count change (Table 7). This confirms that the neural ceiling is K-independent: the pruning tables dominate move ordering regardless of how many endpoints are explored, reinforcing the conclusion that the bottleneck is the classical baseline's strength, not the search budget.

Practical implications. The $\leq$20-move rate—a natural quality threshold given God's Number—rises from 28.0% to 57.5% on the canonical run (roughly doubling), or from a pooled 29% to 57% across the four-seed sweep; in either case this is a large shift in the fraction of solutions that a human speed-cuber would consider "short." Solve time on the Python backend grows from 582 ms to 5103 ms, which is imperceptible for a handful of solves per session; the C backend brings K=5 well under 50 ms for latency-critical uses. For batch applications, the balanced K=3 setting already achieves the highest $\leq$20-rate in our sweep (60.0%) at a lower wall-clock cost than K=5 and is our recommended default.

The neural-LUT result produces a much smaller quality effect (1.13× node reduction, essentially zero wall-clock delta vs. plain K=5) and its value lies less in per-solve speedup than in demonstrating that inference cost is not the bottleneck—a methodological result more than a benchmark-topping one.

6.7 Deployment Recommendations

For most applications, K=3 offers the best trade-off (Table 11): most of the solution-quality gain at a fraction of the cost. The C backend makes even K=5 fast enough for interactive use ($<$100 ms). The neural + LUT row is retained for completeness: it matches plain K=5 in quality and wall-clock within noise, and we do not recommend deploying it over plain K=5 multi-solution search given the small reliability regression (199/200 vs. 200/200) on our benchmark.

7 Conclusion

We extended Kociemba's Two-Phase Algorithm with multi-solution search, neural move ordering, and formal analysis. Rather than asking "can neural methods improve this solver?" in isolation, we map the *boundary conditions* under which learned heuristics add value to a pruning-table-dominated search—and identify a simple algorithmic change (multi-solution search) that delivers the quality gains neural methods target, at zero training cost. Our main findings:

1. **Multi-solution search works.** On the canonical seed-42 run K=5 reduces mean moves by 0.53 ($p < 10^{-20}$, $d_z = 0.79$, 95% CI $[0.44, 0.62]$) and raises $\leq$20-move solutions from 28.0% to 57.5%. A four-seed pooled sweep gives a more conservative $\Delta = 0.40 \pm 0.09$ moves ($d = 0.59$); the improvement is robust and has no failure modes across the configurations we tested.
2. **Representation and label quality drive intrinsic move-prediction accuracy.** Binned coordinate features (161 dims) plus pruning-greedy labels on 2M cubes raise held-out top-1 from $\approx$5.9% to 43.9% (64.5% top-3), using the same small MLP. The end-to-end node-count reduction from this learned ordering over plain K=5 multi-solution search is, however, only $\approx 1.13\times$, and does not translate into a wall-clock speedup in our setting.
3. **Inference overhead is real but solvable.** Per-node neural inference negates the node savings in Python. LUT precomputation (32K entries, 576 KB) largely eliminates this overhead (+2% vs. +22%).
4. **Naïve neural bounds are harmful.** Using neural predictions as IDA* lower bounds causes exponential blowup (Observation 1). The correct integration is move ordering.

Key Takeaways

1. *Multi-solution search is a simple yet effective improvement to Kociemba's algorithm.* A one-parameter change to an off-the-shelf solver buys a statistically robust 0.5-move reduction and roughly doubles the fraction of $\leq$20-move solutions, with no training required.
2. *Representation and label quality—not architecture—drive learned move-prediction accuracy.* Switching from 9 scalar features to 161 binned coordinate features and from proxy labels to pruning-greedy labels moves held-out top-1 from $\approx$5.9% to 43.9% using the *same* small MLP; tripling the parameter count adds only $\approx$2.6pp.
3. *Learned heuristics plateau when the classical heuristic is already strong.* Kociemba's pruning tables leave very little residual ordering signal for a neural predictor to exploit, and our empirical $1.13\times$ ceiling agrees with the order-of-magnitude argument in Observation 2.

4. *LUT distillation solves the inference-cost problem.* Distilling a trained MLP into a 576 KB lookup table drops per-node neural cost to $\approx 1.5\,\mu$s and removes the $+22\%$ wall-clock penalty, demonstrating that learned ordering can be made asymptotically free inside IDA*. The remaining bottleneck is not inference cost but search structure: the underlying $1.13\times$ node reduction is too small to produce a meaningful end-to-end speedup.
5. *Evaluate learned heuristics end-to-end, not by held-out accuracy.* The 43.9% top-1 figure is an order of magnitude above the scalar baseline yet leaves solver performance unchanged; node count and wall-clock time are the metrics that matter for search.

This suggests that future improvements should target pruning-table design or search-space reduction rather than learned move ordering.

Limitations

We separate limitations into practical (properties of our specific implementation and experimental setup) and theoretical (properties of the approach itself).

Practical limitations.

1. *Single-machine, single-threaded evaluation.* All experiments run on a consumer laptop (Apple M-series, 16 GB RAM); multi-core or GPU parallelism is unexploited.
2. *Python-backend measurements.* Absolute wall-clock numbers reflect the pure-Python solver; the C backend is $59\times$ faster, so millisecond-level conclusions should be re-evaluated in production.
3. *Limited scramble diversity.* Benchmarks use 200 cubes per seed across four seeds. While multi-seed results are tight ($\Delta = 0.40 \pm 0.09$), our sample is small relative to the 4.3×10^{19} state space.
4. *Training budget and infrastructure.* Our retrained model uses 2M cubes on a single CPU ($\sim$3 h). Held-out top-1 climbs monotonically with feature resolution and label quality, and the oracle headroom ($\sim 10\times$ in node count) is large, yet our lightweight MLP captures only $1.13\times$. With larger GPU budgets, finer-grained LUTs, and solver-feedback labels in parallel, we expect this ceiling to lift; we report the present numbers as an infra-constrained lower bound.

Theoretical limitations.

1. *Binned-feature resolution.* The LUT quantises each coordinate into 16–32 bins; finer bins approach per-node neural inference at the cost of $O(B^k)$ table size.
2. *Admissibility of neural ordering.* Our use of neural predictions for move ordering is admissibility-preserving; Observation 1 shows why using them as lower bounds is not. The solver depends on Kociemba's pruning tables for correctness.

3. *Phase 2 and MLP coverage.* Both the predictor and LUT are indexed by Phase 1 coordinates only. A three-layer MLP (256/128/64) is deliberately small; whether GNNs on cubie adjacency would yield additional node reduction at the same LUT granularity is open.
4. *Proxy label quality.* Labels are the inverse of the last scramble move rather than the true optimal first move, bounding the predictor's ceiling.

Future Work

1. *Phase 2-aware LUT.* Incorporating Phase 2 coordinates—or building a second, smaller LUT—would address the coverage limitation and is the most immediate extension.
2. *Phase 1 endpoint prediction.* A network trained to predict which G_1 states lead to short Phase 2 solutions would make multi-solution search *smarter* rather than just broader.
3. *Parallel Phase 1 search.* The K endpoints are independent; multi-core or vectorised implementations should give near-linear speedups.
4. *Graph neural networks on cubie adjacency.* Replacing the MLP with a GNN that operates directly on the cube's permutation structure could improve prediction accuracy without inflating parameter count, provided LUT distillation still applies.
5. *Transfer to larger puzzles.* The algorithmic contributions apply unchanged to $4\times4\times4$ and other multi-phase solvers (e.g., Thistlethwaite).
6. *Scaling neural guidance.* Re-running the pipeline on a GPU budget with 10–$100\times$ more data, finer bins (64–128 per coordinate, LUT grows to tens of MB but stays $O(1)$), solver-feedback labels in parallel, and larger models should meaningfully close the gap between the observed $1.13\times$ and the oracle $\sim10\times$ ceiling.

Reproducibility. All benchmarks are reproducible via: `python benchmark.py -scrambles 200 -compare-baseline -max-phase1 5 -seed 42 -v`. Results, models, and scripts are provided.

References

1. H. Kociemba, "Two-Phase Algorithm." [Online]. Available: `https://kociemba.org/cube.htm`. Accessed: Apr. 10, 2026.
2. T. Rokicki, H. Kociemba, M. Davidson, and J. Dethridge, "God's Number is 20," 2010. [Online]. Available: `http://www.cube20.org/`. Accessed: Apr. 10, 2026.
3. muodov/kociemba, "Python package: C and Python implementations of Kociemba's two-phase algorithm." [Online]. Available: `https://github.com/muodov/kociemba`. Accessed: Apr. 10, 2026.

4. F. Agostinelli, S. McAleer, A. Shmakov, and P. Baldi, "Solving the Rubik's Cube with Deep Reinforcement Learning and Search," *Nature Mach. Intell.*, vol. 1, pp. 356–363, 2019.
5. R. E. Korf, "Finding Optimal Solutions to Rubik's Cube Using Pattern Databases," in *Proc. AAAI Conf. Artif. Intell.*, 1997, pp. 700–705.
6. M. Thistlethwaite, "A 45-Move Algorithm for Rubik's Cube," unpublished manuscript, 1981.
7. R. Brunetto and O. Trunda, "Deep Heuristic-Learning in the Rubik's Cube Domain: An Experimental Evaluation," *Proc. ITAT*, pp. 57–64, 2017.
8. W. Shen, F. Trevizan, and S. Thiébaux, "Learning Domain-Independent Heuristics for Grounded and Lifted Planning," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 9883–9891.
9. P. Ferber, F. Geissmann, T. Helmert, *et al.*, "Neural Network Heuristics for Classical Planning: A Study of Hyperparameter Space," in *Proc. Eur. Conf. Artif. Intell. (ECAI)*, 2022, pp. 839–846.
10. A. Felner, R. E. Korf, and S. Hanan, "Additive Pattern Database Heuristics," *J. Artif. Intell. Res. (JAIR)*, vol. 22, pp. 279–318, 2004.
11. J. Fridrich, "CFOP Method." [Online]. Available: `https://www.speedsolving.com/wiki/index.php/CFOP_method`. Accessed: Apr. 10, 2026.
12. R. E. Korf, "Depth-First Iterative-Deepening: An Optimal Admissible Tree Search," *Artif. Intell.*, vol. 27, no. 1, pp. 97–109, 1985.
13. P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.
14. J. Pearl, *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Reading, MA: Addison-Wesley, 1984.
15. J. C. Culberson and J. Schaeffer, "Pattern Databases," *Comput. Intell.*, vol. 14, no. 3, pp. 318–334, 1998.
16. R. E. Korf and A. Felner, "Disjoint Pattern Database Heuristics," *Artif. Intell.*, vol. 134, no. 1–2, pp. 9–22, 2002.
17. D. Silver *et al.*, "Mastering the Game of Go with Deep Neural Networks and Tree Search," *Nature*, vol. 529, pp. 484–489, 2016.
18. D. Silver *et al.*, "Mastering the Game of Go Without Human Knowledge," *Nature*, vol. 550, pp. 354–359, 2017.
19. S. J. Arfaee, S. Zilles, and R. C. Holte, "Learning Heuristic Functions for Large State Spaces," *Artif. Intell.*, vol. 175, no. 16–17, pp. 2075–2098, 2011.
20. J. Schaeffer, "The History Heuristic and Alpha-Beta Search Enhancements in Practice," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 11, no. 11, pp. 1203–1212, 1989.
21. A. Reinefeld and T. A. Marsland, "Enhanced Iterative-Deepening Search," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 16, no. 7, pp. 701–710, 1994.
22. D. Joyner, *Adventures in Group Theory: Rubik's Cube, Merlin's Machine, and Other Mathematical Toys*, 2nd ed. Baltimore, MD: Johns Hopkins Univ. Press, 2008.
23. J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Hillsdale, NJ: Lawrence Erlbaum, 1988.
24. B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman and Hall/CRC, 1993.
25. H. A. David and H. N. Nagaraja, *Order Statistics*, 3rd ed. Hoboken, NJ: Wiley, 2003.
26. G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *Proc. NeurIPS Deep Learn. Workshop*, 2015.
27. D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.
28. E. D. Demaine, M. L. Demaine, S. Eisenstat, A. Lubiw, and A. Winslow, "Algorithms for Solving Rubik's Cubes," in *Proc. Eur. Symp. Algorithms (ESA)*, 2011, pp. 689–700.
29. A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019.
30. M. Zawalski, G. Góral, M. Tyrolski, E. Wiśnios, *et al.*, "What Matters in Hierarchical Search for Combinatorial Reasoning Problems?," in *Proc. ICLR Workshop Generative Models for Decision Making*, 2024.
31. V. Khandelwal, A. Sheth, and F. Agostinelli, "Towards Learning Foundation Models for Heuristic Functions to Solve Pathfinding Problems," arXiv:2406.02598, 2024.